

At Most Half a Run: Guarantees and Limits of Reversal-Based Detection of Order-Dependent Flaky Tests

Anonymous Author(s)

Submission to the ICST 2027 Research Track

Abstract—Should detectors of order-dependent (OD) flaky tests spend their rerun budget on reversed test orders? iDFlakies does, because the reverse of a passing order is more likely to fail. We show that this intuition is correct but its payoff is bounded. First, we prove that an order and its reversal have nonpositively correlated failure events in four structural classes of OD tests; the common-cleaner class, which allows arbitrary nesting, alone covers 259 of the 289 published OD models we study. The proofs combine a composable sum-of-squares certificate, a cyclic-cut reduction to Harris’ inequality, and a two-square window identity; counterexamples mark where the property fails. Second, we prove a stopping rule: independent reversal pairs save at most half a suite run in expected time to a test’s first failure, and independent k -run blocks at most $(k - 1)/2$ runs when $kf \leq 1$, where f is the failure probability. Outcome-gated reversal, as deployed, changes the order distribution and inherits neither guarantee. Simulating the 289 models in 47 modules, pairing saves 0.40 runs per test and adds 0.19–0.33 percentage points at 20 runs. Real executions of three complete modules, one with OD tests that fail in only 3% of orders, match all 40,200 predicted outcomes of 39 OD tests order by order, and reveal OD tests the dataset misses. 4,200 runs of the real iDFlakies confirm the simulated gate rule in all 26,600 of its decisions. Finally, we derive the accuracy a reversal predictor needs to beat random orders.

Index Terms—flaky tests, order-dependent tests, test order, reversal, probabilistic analysis, antithetic sampling

I. INTRODUCTION

An order-dependent (OD) flaky test can pass or fail on the same code depending on which tests run before it [1], [2]. Detectors therefore spend a limited budget executing a suite in different orders [3]–[5]. The engineering question is not just whether a new order is more likely to fail, but *whether choosing it saves enough executions to matter*.

Reversal is a particularly consequential example. Wei et al. [6] argued that reversing a passing order can move a polluter before its victim, and observed a higher conditional failure probability by sampling. iDFlakies adopted reversal when a round finds no new flaky test [6], [7]. This leaves two gaps: a conditional probability gain is not a detection-time saving, and a statement about one passing test does not justify a gate driven by the whole suite.

Our answer is a limit, not a faster detector. Independent reversal pairs save at most *half a suite run per target* in expected time to its first failure, for any fixed failure event on the sampled orders. For a target failing in 1% of random orders, even optimal pairing reduces 100 expected runs only to 99.5, not 50. For independent blocks of k uniform runs,

the saving is at most $(k - 1)/2$ when $kf \leq 1$. Thus a fixed small block cannot yield a constant-factor improvement on increasingly rare failures. This rules out every schedule of independent, bounded-size blocks, reversal pairs included, as a remedy for rare OD tests; it is stronger than observing a small gain on one dataset.

What is new. Reversal and its link to antithetic sampling are old ideas. What was missing is when the antithetic sign actually holds for OD tests, where it can fail (Table II), and what it can buy. We contribute:

- a safety theorem: an order and its reversal have non-positively correlated failures in four structural classes, with a composable sum-of-squares certificate, a cyclic-cut reduction to Harris’ inequality, and a two-square window identity, plus counterexamples at the frontier;
- a limits map: at most half a run for pairs, $(k - 1)/2$ for k -run blocks, lower bounds for broader schedules, and the exact accuracy/cost a reversal predictor needs;
- an evaluation on 289 published OD models in 47 modules, checked against real executions order by order.

What practitioners should do. Keep iDFlakies’ gate at its default budget of 20 runs: there all policies are within 0.5pp of each other, in simulation and in the real tool, and the gate is never significantly worse than random orders. Use unconditional pairing for budgets of two to four runs, where the gate often skips useful reversals. Do not expect more from reversal variants. The misses are rare OD tests, and finding them needs information about which orders will fail, or designs spanning about $1/f$ runs.

Section IV states the theory with proof ideas; complete proofs are in the supplement, and practical readers can skip them. Section V answers three research questions, and Section VI turns them into decisions.

II. BACKGROUND AND MOTIVATION

OD tests. Following Wei et al. [6], a victim v with polluters P and polluter-specific cleaner sets C_p fails in order ω iff

$$\exists p \in P : p \prec_{\omega} v \wedge \nexists c \in C_p : p \prec_{\omega} c \prec_{\omega} v. \quad (1)$$

A brittle fails iff no state-setter precedes it. Li et al. [8] use these semantics in their simulator for 289 OD tests, and they count a test as detected once both a passing and a failing order have been observed. We report this protocol and a known-reference protocol (Section III).

TABLE I
THE THREE-TEST EXAMPLE: CLASSES $\{v, c\}$ AND $\{p\}$.

Order σ	v fails	$R\sigma$	v fails in $R\sigma$
vcp	no	pcv	no
cvp	no	pvc	yes
pvc	yes	cvp	no
pcv	no	vcp	no

The reversal gate in iDFlakies. At the commit we inspected [7], `RandomDetector` requests a fresh shuffled order unless the previous round’s filtered result (new, confirmed flaky tests) is empty. In that case it issues the reverse of the previous order. `TestShuffler` falls back to a fresh order if that reversal has already been issued, so an order is never reversed twice in a row. The source also contains a commented-out alternative that reverses only after a run without any failure. We call the deployed rule `GATE` and the alternative `STRICT`.

A three-test example. Let classes be $\{v, c\}$ and $\{p\}$, where p pollutes v and c cleans. Table I lists all orders. The four legal orders are vcp , cvp , pvc , pcv , and only pvc fails, so the flake rate is $f = 1/4$. The reverse of the passing order cvp is the failing order pvc . Thus $\mathbb{P}(\text{reverse fails} \mid \text{pass}) = 1/3 > f$, which is the effect Wei et al. exploit. Now pair a random order with its reversal. No order fails in both directions, so one pair detects v with probability $2f = 1/2$; two independent orders detect it with probability $7/16$. In expected runs until the first failure, pairing needs 3.5 runs and independent sampling needs 4. The saving is exactly half a run, and Theorem 2 shows that half a run is the most independent reversal pairing can save.¹

III. MODEL

Definition 1 (Hierarchical legal orders). A rooted tree has the tests as leaves. At every internal node, the children are permuted uniformly and independently of other nodes, and each child’s subtree runs contiguously. The induced distribution μ on the set Ω of legal orders is uniform. The map R reverses a whole order; it corresponds to reversing every node’s permutation, so R is a measure-preserving involution.

Class-contiguous orders form the two-level case (root $\rightarrow$ classes $\rightarrow$ methods). Deleting tests that do not affect a target keeps the induced order uniform on the pruned tree, so only relevant tests matter.

Common-cleaner semantics. Fix a victim v with pairwise disjoint sets P and C . Every run starts with a clean bit. Polluters set the bit, cleaners clear it, and v fails iff the bit is set when v runs. This is Eq. (1) with $C_p = C$ for all p .

¹The closed form printed for this setting (“Special Case 2” in [6, Sec. 4.4]) gives $\mathbb{P}(\text{pass} \wedge \text{reverse fails}) = 1/2$ here ($k = 2$, $\pi_1 = 0$, $\gamma_1 = 1$, $\pi_2 = 1$, $\gamma_2 = 0$) instead of $1/4$. In general it overstates the probability by $\gamma_1 S_\pi / [k(\pi_1 + \gamma_1 + 1)]$ in the notation of [6]. The reported comparison in [6] was obtained by sampling. Corrected formulas and an exhaustive check are in our package.

Target statistics. For a target, let $F = \{\sigma : \text{target fails in } \sigma\}$ and

$$f = \mu(F), \quad B = \mathbb{P}(F(\sigma) \wedge F(R\sigma)), \\ q = 1 - 2f + B, \quad d = f^2 - B.$$

B is a joint probability; the covariance of the two failure indicators is $-d$. Because R preserves μ , the order and its reversal both fail with probability f , and q is the probability that both pass.

Detection protocols. With a *known passing reference* (KR), a passing outcome is supplied externally; this is an assumption, not a model-consistency claim about the recorded original order, and a target is detected at its first failing run. With *no reference* (NR), the definition of Li et al. [8], both a failing and a passing run must occur within the budget.

Schedules. A *run* is one execution of the complete suite (iDFlakies calls it a round), and a *detector run* is one sequence of r runs. A budget of r runs is spent by IID (fresh uniform orders), PAIR ($\sigma_1, R\sigma_1, \sigma_2, R\sigma_2, \dots$ with independent σ_i), GATE, or STRICT (Section II). We write $r = 2m + \epsilon$ with $\epsilon \in \{0, 1\}$.

IV. GUARANTEES AND LIMITS

A. When is pairing safe?

Theorem 1 (Safety of independent reversal pairs). *Under the legal-order distribution of Section III, $B \leq f^2$ in each of the following cases:*

- (a) *Arbitrary nested groupings with a common cleaner set, disjoint from the polluters; or a brittle with at least one state-setter (then $B = 0$).*
- (b) *Unconstrained orders with arbitrary polluter-specific cleaners disjoint from the polluters.*
- (c) *Class-contiguous orders with disjoint roles, where no class that contains a polluter, the victim’s class included, contains a cleaner of any polluter outside that class.*
- (d) *Class-contiguous orders with disjoint roles, where every non-global cleaner of a polluter lies in that polluter’s class; a global cleaner cleans every polluter.*

For each such target, independent reversal pairs have detection probability at least that of independent random orders at every budget, both with and without a passing reference. Their expected number of detected targets is also no smaller.

Proof ideas; complete proofs in the supplement. Scan outward from v on each side; a side *fails* if it meets an active polluter. Case (a) is immediate for brittles, since a state-setter lies on one side of v .

(a) *A composable certificate.* For a subtree $T \ni v$, let u_T be the probability that T has no relevant test left of v , and t_T the probability of this together with right failure. Put $G_T(x) = (f_T + u_T x)^2 - B_T - 2t_T x$. At the lowest node with $k \geq 2$ relevant children, let the $k - 1$ siblings of v ’s child have probabilities α_i that their first relevant test, seen from an end, is a polluter; let $S = \sum \alpha_i$ and $Q = \sum \alpha_i^2$.

Uniform permutations give $f = S/k$, $B = (S^2 - Q)/[k(k-1)]$, $u = 1/k$, and $t = S/[k(k-1)]$, so

$$G(x) = \frac{1}{k^2} \left(x - \frac{S}{k-1} \right)^2 + \frac{Q - S^2/(k-1)}{k(k-1)} \geq 0.$$

The second term is nonnegative by Cauchy–Schwarz. At an ancestor with k relevant children whose other children have α -sum S , a sibling can only decide a side that T leaves open. Hence, with $h = S/k$, we get $f' = f + uh$, $B' = B + 2th$, $u' = u/k$, and $t' = t/k$, that is, $G_{\text{parent}}(x) = G_T(h + x/k)$. Nonnegativity on $\mathbb{R}$ survives this affine substitution, and $x = 0$ at the root gives $f^2 - B \geq 0$.

(b) *Cyclic cut*. Write $\sigma = A v C$ and map it to $T\sigma = C v A$, a measure-preserving bijection that commutes with reversal. In σ , a polluter $p \in A$ is active iff every cleaner of p in A precedes p . In $T\sigma$, this means that p comes after v and after all of its cleaners. With i.i.d. uniform heights Z , $F(\sigma)$ therefore becomes $M_+ = \{\exists p : Z_p > Z_t \ \forall t \in \{v\} \cup C_p\}$, and $F(R\sigma)$ becomes M_- , the same event with “ $<$ ”. With disjoint roles, M_+ increases in the polluters’ heights and decreases in all others, and M_- does the opposite. Harris’ inequality [9] gives $B = \mathbb{P}(M_+ \cap M_-) \leq f^2$.

(c) *Conditional monotonicity*. Use i.i.d. uniform class and method times, and condition on the victim’s times (u, w) . Order all other coordinates cyclically away from the victim. Then left failure is increasing and right failure is decreasing, so Harris’ inequality gives $B \leq \iint a(u, w) a(1-u, 1-w)$, where a is the conditional failure probability. One-sidedness makes a monotone in each coordinate, and Harris’ inequality on $[0, 1]^2$ bounds the integral by f^2 .

(d) *Window identity and composition*. Classify each side by its first decisive event: fail, clean (a global cleaner), or undecided. For the victim’s class alone, let u_0, t_0, z_0 be the probabilities that the left side is undecided, that it is undecided while the right side fails, and that both sides are undecided. Put $H_0(x) = (f_0 + u_0x)^2 - B_0 - 2t_0x - z_0x^2$. Condition on the positions of v and the g global cleaners in its class. The windows they cut are adjacent uniform spacings whose law does not depend on v ’s rank among them. For $g \geq 1$, this gives $u_0 = (1 - f_0)/(g+1)$, $t_0 = (f_0 - B_0)/(g+1)$, and $z_0 = 0$, hence

$$H_0(x) = d_0 \left(1 - \frac{x}{g+1} \right)^2 + q_0 \left(\frac{x}{g+1} \right)^2 \geq 0 \quad \text{for all } x \in \mathbb{R},$$

with $d_0 = f_0^2 - B_0 \geq 0$ by case (b). For $g = 0$, $H_0(x) = d_0(1-x)^2$. Because non-global cleaners are class-local, the other classes form an independent ensemble of fail/clean/transparent items with its own $H_E \geq 0$. Resolving undecided sides by the ensemble gives the exact identity $H(x) = H_0(h + ax) + z_0 H_E(x)$, where h and a are the ensemble’s failure and transparency probabilities, and $f^2 - B = H(0) \geq 0$. $\square$

The invariant $H \geq 0$ on all of $\mathbb{R}$ says that every scoring of the three side states has nonpositive left–right covariance. Because it composes, it also lifts along the victim’s path in deeper hierarchies (supplement).

TABLE II
SCOPE OF PAIRING SAFETY ($B \leq f^2$). LETTERS REFER TO THEOREM 1. “OPEN” IS NOT USED BY THE EMPIRICAL CONCLUSIONS.

Order structure	Proven scope and boundary
Unconstrained	Common or arbitrary specific cleaners with disjoint roles: (a), (b).
Class-contiguous	Common cleaners: (a); specific cleaners satisfying (c) or (d). General disjoint case open; overlapping roles can fail (7 tests).
Deeper nesting	Common cleaners: (a). Specific cleaners can fail (6 tests; Example 2).

What safety means. For $0 < f < 1$, the conditional chance that a passing order’s reversal fails is $(f - B)/(1 - f) \geq f$. An independent pair misses with probability $q = 1 - 2f + B \leq (1 - f)^2$. Multiplying over independent pairs proves detection dominance; linearity gives the expected-count result without assuming independence between targets. The guarantee is per target: it does not cover the time to find every OD test, the chance of finding at least one, or a suite-level gate.

The open class-contiguous case has neither condition (c) nor (d). Exhaustive search found no counterexample among 578,658 instances with at most seven tests; this is not a proof. All dataset models already satisfy proved cases, so resolving the conjecture is not needed for our conclusions.

B. A map of what scheduling can buy

Computing detection at a budget. For $r = 2m + \epsilon \geq 1$ runs,

$$\begin{aligned} D_{\text{PAIR}}^{\text{KR}} &= 1 - q^m(1 - f)^\epsilon, & D_{\text{IID}}^{\text{KR}} &= 1 - (1 - f)^r, \\ D_{\text{PAIR}}^{\text{NR}} &= D_{\text{PAIR}}^{\text{KR}} - B^m f^\epsilon, & D_{\text{IID}}^{\text{NR}} &= D_{\text{IID}}^{\text{KR}} - f^r. \end{aligned} \quad (2)$$

Without a reference, subtract the probability that every run fails, since observing both outcomes is required.

Theorem 2 (Block limit). *Let a schedule consist of independent blocks of k runs. Each run is uniformly distributed, and dependence within a block is arbitrary. Let T be the number of runs until a target’s first failure, with $0 < f < 1$. If $kf \leq 1$, then*

$$\mathbb{E}T \geq \frac{1}{f} - \frac{k-1}{2},$$

with equality iff the failure events within each reached block are pairwise disjoint. For reversal pairs ($k = 2$), $\mathbb{E}T_{\text{PAIR}} = (2 - f)/(2f - B)$ exactly, and

$$0 \leq \mathbb{E}T_{\text{IID}} - \mathbb{E}T_{\text{PAIR}} = \frac{f^2 - B}{f(2f - B)} \leq \frac{1}{2}$$

when $B \leq f^2$, with equality iff $B = 0$.

Proof. Let $d_i \leq f$ be the probability that a block’s first failure is its $(i+1)$ -st run, and let $h = \sum_i d_i$. For identically distributed blocks, summing survival probabilities gives $\mathbb{E}T = [k(1 - h) + \sum_i (i+1)d_i]/h$. For $h = \rho f$, the sum $\sum_i (i+1)d_i$ is smallest when mass f sits on the earliest runs, so

it is at least $f\rho(\rho+1)/2$. Hence $\mathbb{E}T \geq k/(\rho f) - k + (\rho+1)/2$. This is decreasing in $\rho \in [0, k]$ because $\rho^2 \leq k^2 \leq k/f$, and $\rho = k$ gives the bound. Equality forces $h = kf$ and $d_i = f$ for all i , i.e., disjoint failure events. The supplement extends the bound to independent blocks with different laws by weighting each block's expected cost by its reach probability. For pairs, $\mathbb{P}(T > 2j) = q^j$ and $\mathbb{P}(T > 2j+1) = q^j(1-f)$, which gives $(2-f)/(1-q)$; the bound $\leq \frac{1}{2}$ rearranges to $B(2-f) \geq 0$. $\square$

In the $kf \leq 1$ regime, independent blocks of bounded size thus buy at most $(k-1)/2$ runs. A test with $f = 0.01$ needs at least 99.5 runs on average under PAIR, not 50. Within this block family, a constant-factor reduction requires blocks of order $1/f$. Information-directed nonuniform scheduling is a different way to leave the family; the next theorem gives two benchmarks beyond it.

Theorem 3 (Lower bounds beyond independent blocks). *For a target with $0 < f < 1$:*

- (a) *Any schedule whose runs are individually uniform, with arbitrary dependence, satisfies*

$$\mathbb{E}T \geq L(f) := (N+1)(1-Nf/2), \quad N = \lfloor 1/f \rfloor.$$

- (b) *Any policy issuing i.i.d. fresh orders in sequence or reversals of already issued fresh orders satisfies*

$$\mathbb{E}T \geq O(f, B) := \frac{1+f-B}{2f-B}.$$

This includes adaptive policies and policies with an oracle. An oracle that reverses a passing fresh order exactly when its reversal fails attains (b) for that target.

Proof ideas. For (a), the union bound gives $\mathbb{P}(T > t) \geq (1-tf)^+$; summing yields $L(f)$. For (b), let i^* be the first fresh order whose reversal orbit contains a failure. It is geometric with parameter $2f-B$. Every policy must issue at least i^* fresh orders, plus one run if that first useful order passes, giving $[1+(f-B)]/(2f-B)$. The supplement proves both results, including target-aware attainment of (a). $\square$

Interpretation: a benchmark, not a delivered speedup. For rare tests with $B = 0$, both bounds are about $(1/f+1)/2$, far below $1/f - 1/2$ for pairing. But these are *per-target* benchmarks: a target-aware optimum need not be achievable by a single schedule serving all targets. Their average bounds the average detection time from below. Moreover, (a) and (b) constrain different classes: gates need not have uniform marginals, and arbitrary nonuniform failure-directed schedules need satisfy neither bound.

Without a passing reference. Let $T^\pm$ count runs until both outcomes occur. The supplement proves

$$\mathbb{E}T_{\text{PAIR}}^\pm = \frac{2-f}{2f-B} + \frac{1+f}{1-B} - 1, \quad \mathbb{E}T_{\text{IID}}^\pm = \frac{1}{f} + \frac{1}{1-f} - 1. \quad (3)$$

When $B \leq f^2$, pairing saves between zero and one run; one run is attained only at $f = 1/2$, $B = 0$. Apply the half-run bound separately to the first failure and the first pass.

Rare failures gain little at any budget. Under KR, $\Delta_r = D_{\text{PAIR}} - D_{\text{IID}}$ obeys

$$0 \leq \Delta_r \leq \lfloor r/2 \rfloor (f^2 - B)(1-f)^{r-2}.$$

For $0 < f \leq 1/2$, its budget-uniform upper envelope is $f^2/[-2e(1-f)^2 \ln(1-f)] = f/(2e) + O(f^2)$. The leading term is asymptotic, not an exact finite- f upper bound. This explains why a large conditional probability improvement can produce a small absolute detection gain.

C. What does not transfer to the deployed tool

Proposition 1 (Gated reversal changes the order law). *Let $\sigma \sim \mu$. With probability $g(\sigma)$ run $R\sigma$ next, and otherwise run a fresh $\eta \sim \mu$. The law ν of the second run satisfies $\frac{d\nu}{d\mu}(\tau) = 1 + g(R\tau) - \mathbb{E}_\mu g$. Hence $\nu = \mu$ iff g is almost surely constant. A deterministic gate that accepts a set of mass a moves the law by total variation $a(1-a)$.*

Proof. The reversal branch contributes $\int g(\sigma) \mathbf{1}\{R\sigma \in A\} d\mu = \int_A g(R\tau) d\mu(\tau)$, and the fresh branch contributes $(1 - \mathbb{E}g)\mu(A)$. $\square$

The union bound $D \leq \min(1, rf)$ holds for any schedule whose runs are individually uniform, but it cannot be used for gated schedules without their actual marginals. The same applies to per-target arguments. GATE conditions on *other* tests' outcomes, not on the target's own pass. Two targets with identical (f, B) can therefore behave differently under the gate, and no function of per-target statistics determines a gated suite's outcome.

Example 1 (The gate can lose to random). Two bits A, B start clear. Classes $\{v_A, p_B\}$, $\{v_B, p_A\}$, and $\{c\}$ contain victims that check their bit, polluters that set the other class's bit, and a cleaner c that clears both. Each victim satisfies the common-cleaner model. Of the 24 legal orders, 8 pass both victims; this set is closed under reversal. The remaining 16 orders fail exactly one victim, and reversal swaps which one. With two runs under KR, the expected numbers of detected victims are 10/9 for IID, 8/9 for the initial GATE step, and 4/3 for PAIR. For at least one detection, however, IID (8/9) beats PAIR (2/3).

Neither rule dominates the other. In the module with classes $\{v_0, v_1, c\}$ and $\{p\}$, where p pollutes both victims and c cleans only for v_1 , the gate detects 25/16 victims in two runs and pairing 3/2.

Example 2 (Three levels break the guarantee). For the tree $((v, (p_1, c_1)), (p_2, c_2)), p_3)$ with $C_{p_1} = C_{p_2} = \{c_1\}$ and $C_{p_3} = \{c_2\}$, all 32 legal orders give $f = 11/16$ and $B = 1/2$, so $B - f^2 = 7/256 > 0$. With two runs, pairing detects with probability 7/8 and IID with 231/256. The safety theorem therefore does not extend to nested groupings with polluter-specific cleaners (Table II). The limits of Section IV-B that do not use $B \leq f^2$ remain valid.

What a useful predictor must achieve. The oracle benchmark assumes perfect information. To make it actionable,

consider a fixed predictor for one target: after each passing fresh order, it either requests its reversal or skips to a fresh order. Fresh-order/predictor cycles are independent. Let s be its true-positive rate on pass/fail pairs and a its false-positive rate on pass/pass pairs. Put $J = f - B$ and $q = 1 - 2f + B$. A renewal calculation gives

$$\mathbb{E}T_{s,a} = \frac{1 + sJ + aq}{f + sJ}, \quad (4)$$

$$\mathbb{E}T_{s,a} < 1/f \iff sJ(1 - f) > afq.$$

A cycle costs one fresh run plus a reversal with probability $sJ + aq$, and succeeds with probability $f + sJ$. This proves the formula. The full derivation, edge cases, and exact checks are in the supplement.

For a hypothetical target with $f = 0.01$, $B = 0$, a predictor with $s = 0.8$ and $a = 0.1$ needs 61.44 expected runs, versus 100 for random sampling, 99.5 for pairing, and 50.5 for the oracle. This is a calculated design point, not an evaluated predictor. If prediction costs c suite-run equivalents per fresh order, replace the numerator by $1 + c + sJ + aq$; the break-even condition becomes $sJ(1 - f) > f(c + aq)$. This gives predictor research a measurable target; a suite-level predictor must also balance conflicting targets.

Verification scope. Complete proofs, the unresolved frontier, and auxiliary identities are in the supplement. Exact checks enumerate 14,791 common-cleaner hierarchy models and 731,630 legal orders, check closed forms and small counterexamples, and test the scheduling identities.

V. EMPIRICAL STUDY

The theory says what pairing can and cannot buy. We now measure it on real OD tests, with three questions:

- **RQ1:** How much does reversal pairing buy for real OD tests, measured against the limits of Section IV?
- **RQ2:** Does iDFlakies’ outcome-gated reversal change this picture for whole modules?
- **RQ3:** Does the polluter/cleaner abstraction behind RQ1 and RQ2 predict real executions?

A. Subjects and Method

We use the public dataset of Li et al. [8]: 289 known OD tests (231 victims and 58 brittles) in 47 modules of 27 open-source Java projects. It gives polluters and cleaners, or state-setters, for every test, and each module’s recorded original order. Our primary semantics **S1** is exactly that of Li et al.’s reference simulator: Eq. (1), with the 21 cleaner *groups* (“ $a|b$ ”) split into individual cleaners. The sensitivity semantics **S2** lets a group clean only if all of its members run between polluter and victim. Orders follow iDFlakies’ default: a uniform class order and uniform method orders.

Per-test statistics. 258 tests (all brittles, and victims whose polluters share one cleaner set) have exact rational f and B from two-level closed forms. For the other 31 victims we estimate f and B from 2×10^6 sampled orders each and propagate uncertainty by a parametric bootstrap. Gains and expected runs then follow from Eqs. (2)–(3) and Theorem 2.

TABLE III
THE LIMITS MAP ON REAL DATA: EXPECTED RUNS TO THE FIRST FAILURE (KR), AVERAGED OVER ALL 289 OD TESTS AND OVER THE 71 RARE ONES ($f < 0.1$). FOR $B = 0$ BOTH THE ORACLE AND THE UNIFORM-MARGINAL BOUND ARE CLOSE TO $(1/f + 1)/2$, SO THEIR AVERAGES NEARLY COINCIDE.

Schedule (theorem)	all	rare
IID ($1/f$)	10.22	34.66
PAIR, outcome-blind (Thm. 2, exact)	9.82	34.16
4-run blocks (lower bound)	9.45	33.16
reverse-or-fresh (per-target oracle)	5.62	17.83
uniform marginals (lower bound)	5.62	17.83

Suite simulation. For RQ2 we simulate all targets of a module jointly on the same orders, 100,000 detector runs of 40 rounds per module and policy. GATE reproduces the tool’s rule: reverse iff the previous round detected no new target and the previous order was not itself a reversal. All policies consume the same stream of fresh orders (common random numbers); per-module contrasts use a Bonferroni correction over 47 modules. We do not model confirmation reruns, unknown flaky tests, or wall-clock cost. The simulator agrees with the per-target closed forms (largest $|z| = 3.26$ over 6,272 comparisons) and with the unmodified simulator of Li et al. on all 289 tests.

Reference outcome. Under the abstraction, the recorded original order fails for 108 targets, although it was recorded as passing. KR supplies an external passing reference regardless; NR needs none. We report all targets. Restricting the scoring to the 181 consistent targets changes no conclusion (RQ2), and RQ3 shows that such failures can be real.

B. RQ1: What Pairing Buys per Test

Pairing is safe for every model. Theorem 1(a) alone covers 259 of the 289 models: all brittles, 200 victims whose polluters share their cleaners, and one victim whose only polluter is listed as its own cleaner, a vacuous entry. The remaining 20 and 10 victims satisfy the class-contiguous cases (c) and (d). Independently of the theorems, $d = f^2 - B$ is positive for every model: exactly for the 258 closed-form tests, and with pointwise 95% intervals for the other 31. So pairing never lowers any test’s detection probability, and no dataset model reaches the boundary of Example 2.

But it saves less than half a run. 211 tests have $B = 0$ and save exactly half a run (Fig. 1(a)). On average pairing saves 0.40 runs: 10.22 under IID versus 9.82 under PAIR (NR: 0.70 runs). At fixed budgets the average per-test gain is 11.18 pp at $r = 2$, but only 0.46 pp at $r = 10$ and 0.19 pp at $r = 20$ (KR). The gain follows the rare-failure bound of Section IV closely (Fig. 1(b)): it peaks near $f = 1/r$ and vanishes for frequently failing tests, which are found anyway. Reversal thus already delivers nearly all that independent pairs can deliver.

The misses, and the headroom, are rare tests. At $r = 10$ and $r = 20$, 94% and 98% of the IID misses come from the 71 tests with $f < 0.1$. Most are brittles and single-polluter victims: they have $B = 0$, but fail rarely. Table III places

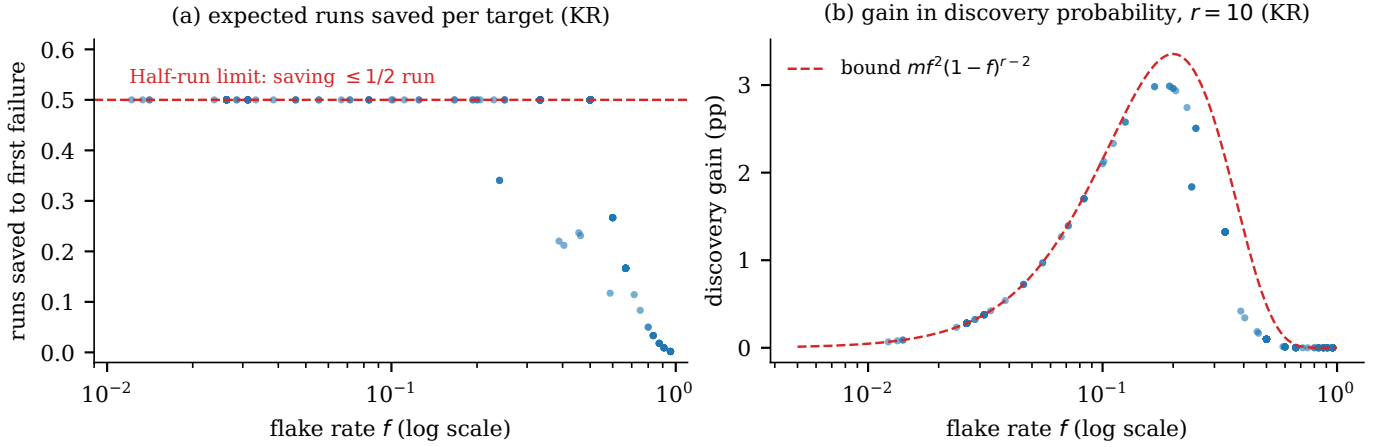


Fig. 1. Per-test benefit of independent reversal pairs versus flake rate (KR, S1). (a) Expected runs saved until the first failure: 211 tests sit exactly on the half-run limit of Theorem 2. (b) Gain in detection probability after 10 runs against the rare-failure bound in Section IV; most tests lie on the bound because $B \approx 0$.

TABLE IV
SUITE-LEVEL DETECTION (% OF 289 OD TESTS) AND DIFFERENCES IN PERCENTAGE POINTS WITH 95% CI HALF-WIDTHS (S1).

Budget r (runs)	2	4	10	20	40
<i>No reference (ISSTA'23 definition)</i>					
IID	32.5	57.9	75.5	84.0	91.7
Pair	54.9	64.9	76.4	84.4	91.9
Gate (iDFlakies)	54.9	64.4	76.4	84.5	92.0
Strict gate	34.8	59.2	75.8	84.2	91.8
Δ Pair–IID	$+22.40 \pm 0.04$	$+6.94 \pm 0.03$	$+0.85 \pm 0.02$	$+0.33 \pm 0.02$	$+0.18 \pm 0.01$
Δ Gate–IID	$+22.40 \pm 0.04$	$+6.48 \pm 0.03$	$+0.86 \pm 0.02$	$+0.43 \pm 0.02$	$+0.24 \pm 0.01$
Δ Pair–Gate	$+0.00 \pm 0.00$	$+0.46 \pm 0.01$	-0.01 ± 0.00	-0.10 ± 0.00	-0.06 ± 0.01
<i>Known passing reference</i>					
IID	58.0	71.2	81.2	86.9	92.8
Pair	69.1	74.8	81.7	87.1	92.9
Gate (iDFlakies)	60.3	73.3	81.6	87.1	92.9
Strict gate	60.3	72.5	81.5	87.1	92.8
Δ Pair–IID	$+11.19 \pm 0.04$	$+3.54 \pm 0.02$	$+0.46 \pm 0.01$	$+0.19 \pm 0.01$	$+0.13 \pm 0.01$
Δ Gate–IID	$+2.31 \pm 0.02$	$+2.09 \pm 0.02$	$+0.39 \pm 0.01$	$+0.17 \pm 0.01$	$+0.11 \pm 0.01$
Δ Pair–Gate	$+8.88 \pm 0.03$	$+1.45 \pm 0.01$	$+0.07 \pm 0.00$	$+0.02 \pm 0.00$	$+0.02 \pm 0.00$

the averages on the map of Section IV. The uniform-marginal bound lies 45% below IID, and pairing closes 8.6% of that gap. For the rare tests, IID needs 34.66 runs and the bound is 17.83. Per-target optima may conflict, so the gap bounds what any detector could save rather than offering a saving. Reducing a rare test’s expected runs by a fixed fraction η with independent blocks requires $k \geq 1 + 2\eta/f$ runs per block (Theorem 2).

Answer: Pairing is safe for all 289 models, and saves 0.40 runs per test on average, below 1 pp of detection from 10 runs on. The remaining headroom lies with rare tests and needs longer-horizon or information-directed designs.

C. RQ2: The Deployed Gate

Table IV compares the four policies on complete modules.

Differences are confined to tiny budgets. Under KR, PAIR improves on IID by +11.19 pp at $r = 2$ and +3.54 pp at $r = 4$,

but only by +0.46 pp at $r = 10$ and +0.19 pp at $r = 20$ (NR: +22.40, +6.94, +0.85, +0.33 pp).

The gate is never significantly worse than random orders. This holds in every module, under both protocols, at every budget from 1 to 40 runs; the most negative module-level z -score is -1.0 before any multiple-comparison correction. This is a non-detection of harm, not an equivalence test. From 10 runs on, GATE and PAIR differ by at most 0.11 pp in aggregate, with signs that vary by module. Under NR the gate almost always reverses after the first round, because one run cannot reveal both outcomes.

In simulation, the gate costs detection only with a tiny budget and a passing reference. There the first round often finds something new, so GATE skips the reversal and trails PAIR by +8.88 pp at $r = 2$. STRICT is weaker still, because a run without any failure is rare in modules with many OD tests. Per module (supplement), reversal adds at most about 1 pp at $r = 10$ under KR. The largest effects (+3.0 pp, NR) occur where OD tests fail in most orders, so that detection waits for a *passing* run.

The real tool. We also ran the unmodified iDFlakies (commit f54b3f0) on the two frequently failing executed modules of RQ3, together with two minimal patches of its decision code: one never reverses (IID), and one reverses after every fresh order (PAIR). Every variant made 20-round detector runs with shared seeds (1,000 per variant on `aismessages`, 400 on `http-request`; 4,200 runs in total), under a protocol frozen before the first run. The simulation matches the tool. All 84,000 executed orders were class-contiguous permutations of the module. The gate reversed exactly when our rule predicts, in all 26,600 decisions. All 720,000 target outcomes on the tool’s own orders matched S1. Table V gives the measured contrasts. On `aismessages` they reproduce the simulation (e.g., GATE–IID at $r = 2$: +6.1 pp measured, +5.7 pp simulated). On `http-request`, the reference order that the tool reconstructs fails 15 of the 25 targets, exactly as S1 predicts for that order. Native Maven passes them, so this

TABLE V

REAL IDFLAKIES RUNS: DETECTION DIFFERENCE IN PERCENTAGE POINTS OF THE MODULE’S TARGETS (KR, COMPUTED FROM PER-ROUND TARGET OUTCOMES; MEAN AND 95% BOOTSTRAP CI OVER PAIRED DETECTOR RUNS).

Module	Contrast	$r = 2$	$r = 4$	$r = 10$	$r = 20$
aismessages	GATE—IID	+6.1 [+4.9,+7.5]	+1.2 [+0.8,+1.8]	+0.0 [+0.0,+0.0]	+0.0 [+0.0,+0.0]
	PAIR—GATE	+5.0 [+4.1,+6.0]	+0.0 [+0.0,+0.0]	+0.0 [+0.0,+0.0]	+0.0 [+0.0,+0.0]
http-request	GATE—IID	+0.0 [+0.0,+0.0]	+0.4 [-0.5,+1.3]	+1.0 [+0.7,+1.4]	+0.1 [+0.0,+0.1]
	PAIR—GATE	+12.5 [+9.8,+15.1]	+9.0 [+7.1,+10.8]	+0.6 [+0.3,+0.9]	+0.0 [+0.0,+0.0]

order is not the one Maven executed; we did not diagnose the difference. Round-0 *passes* of those targets then count as new detections, so the gate never reverses in round 1 and forgoes most of reversal’s benefit at small budgets. This is the suite-level dependence of Section IV-C, observed in the deployed tool.

Robustness. Under S2 the suite contrasts change by at most 0.04 pp at any budget. Scoring only the 181 reference-consistent targets gives PAIR—IID at $r = 10$ of +0.62 pp (KR) and +0.86 pp (NR). Weighting modules equally gives +0.73 pp and +1.21 pp. The supplement has the tables.

Answer: In simulation, all reversal policies are within 1 pp of random orders from 10 runs on, and the deployed gate is never significantly worse. The real tool follows the simulated gate rule exactly; in its two single modules, differences reach 1.7 pp at $r = 10$ and vanish by $r = 20$ (≤ 0.06 pp). At budgets of two to four runs, unconditional pairing is better, especially when the tool’s reference order does not pass.

D. RQ3: Real Executions

RQ1 and RQ2 rest on the published polluter/cleaner models. We therefore executed complete modules and compared each real outcome with the model’s prediction for the same order.

Protocol. Before building anything, we froze the modules, fallbacks, seeds, sample sizes, and stopping rules. Selection could not depend on model agreement. The primaries were aismessages (two targets, model $f = 2/3$, $B = 1/3$) and http-request/lib (25 targets, six reference-inconsistent, $f = 1/3$, $B = 0$). Both built at the dataset revisions without changes to code, tests, dependencies, or fixtures, and their native test inventories matched the recorded ones exactly (44 and 163 tests). Every execution ran the complete module in a fresh JVM with normal class and method fixtures. An ordered JUnit runner kept whole-class execution and checked each actual test start against the requested order. The formal sample has 300 independent uniform class-contiguous orders per module, each followed by its reversal. All 1,200 formal executions were valid, with exact order agreement and no skips or timeouts. 26 pilot runs (three repetitions of the recorded order, and five random orders twice each) were stable and are excluded.

Rare failures. Both modules have $f \geq 1/3$, whereas our argument concerns rare failures. A second protocol, frozen before any build, therefore added marine-api (926 tests, 12 targets, model $f = 1/32$, $B = 0$) with 1,000 pairs and a fixed fallback order. It built unchanged, and no fallback was

TABLE VI

REAL EXECUTIONS: COMPLETE-SUITE SIZE / OD TARGETS, INDEPENDENT PAIRS, RANGES OF OBSERVED ANCHOR f AND JOINT B ACROSS TARGETS, AND S1 MISMATCHES / TARGET OUTCOMES. THESE RANGES ARE NOT CONFIDENCE INTERVALS.

Module	tests/OD	pairs	$\hat{f}$	$\hat{B}$	mismatch
aismessages	44/2	300	0.673	0.293–0.307	0/1200
http-request	163/25	300	0.313–0.407	0.000	0/15000
marine-api	926/12	1000	0.028–0.030	0.000	0/24000

needed. Before the formal sample, the runner gained method-order control for one JUnit 3 class, and it now accepts the one natively ignored test; the package documents both. All 2,000 formal executions were valid.

Every outcome matches. All 40,200 target outcomes matched S1 and S2 order by order (Table VI), which is stronger than matching failure rates. In marine-api, some target failed in only 86 of 2,000 executions, and the model predicted each of these failures and every pass. With the pair as the independent unit, zero mismatching pairs bound the mismatch probability (one-sided 95%) by $1 - 0.05^{1/300} \approx 0.0099$ in the first two modules and by 0.0030 in marine-api. There the model’s $f = 1/32$ lies inside every target’s 95% interval (observed 0.028–0.030), and no pair failed in both directions. The 12 targets fail in correlated groups, so they are not 12 independent confirmations. For 2 of the 27 targets in the first two modules, the model’s theoretical f lies outside the observed 95% interval, but the model has the identical rate on the same sampled orders; this is sampling variation, not disagreement.

The recorded original order really fails. Native Maven runs passed every test, but executing the recorded original order failed the six reference-inconsistent http-request targets in all three repetitions, exactly as S1 and S2 predict. For these targets the model is right and the historical passing reference does not reproduce.

The dataset misses OD tests. Three http-request tests outside the published targets also failed in about a third of the orders. They behave exactly like the module’s 25 known victims. The same polluter and cleaner, which set and reset a static connection factory, explain all 613 recorded runs and 14 targeted confirmation runs. We report them but keep them out of the target set.

Answer: The abstraction predicts every sampled outcome of the 39 executed targets exactly, in the rare regime as well as for frequently failing and reference-inconsistent targets. It also explains three OD victims that the dataset missed.

VI. DISCUSSION

For tool builders: keep the gate, and stop tuning reversal. At idFlakies’ default of 20 runs, the gate, unconditional pairing, and random orders differ by at most 0.5 pp in simulation and in the real tool, and the gate is never significantly worse than random. Only for small per-commit budgets does unconditional pairing pay off: by +8.88 pp over the gate at

$r = 2$ in simulation, and by up to 12.5 pp in the real tool when its reference order does not pass. Beyond that, no reversal variant can help much. Independent pairs save at most half a run per test, and blocks of k runs at most $(k - 1)/2$. These limits need only uniform marginals, not our OD model, so they hold whatever the tests do.

For rare OD tests: buy information, or coordinate longer horizons. Rare tests cause almost all misses, and the dataset’s OD tests were themselves found by earlier detectors, so rare ones are probably underrepresented. Equation (4) tells a reversal predictor what it must achieve: $sJ(1 - f) > f(c + aq)$. Shared-state analysis [10]–[12] and prioritisation [13] are candidate sources of that information. Covering designs over about $1/f$ runs [6], [8] are the other way past the block limit.

For evaluators: separate model, simulator, and execution. Agreement with a reference simulator checks only the simulator; real executions check the model. Evaluate outcome-gated policies jointly over a module, because no per-target statistic determines them (Proposition 1), and state the passing-reference assumption.

VII. THREATS TO VALIDITY

Construct. The 47-module results rest on the polluter/cleaner abstraction of prior work [6], [8]. RQ3 checks it against real executions, but only for the sampled orders of the executed modules. The dataset lists no polluter groups, so OD tests that need several polluters jointly are not represented. We ignore non-OD flakiness, failures that alter later behaviour, confirmation reruns, the tool’s reuse of historical orders, and wall-clock cost. Our detection counts any failing (KR) or passing-and-failing (NR) outcome, not iDFlakies’ confirmed-flaky classification; for the real tool, the package reports both. Under S1, 108 of the 289 targets fail their recorded original order. KR assumes a passing reference regardless, and the real tool shows that its own reference need not pass.

Population. The dataset’s OD tests were found by earlier detectors, so rarely failing OD tests are probably underrepresented, and RQ3 found three victims it misses. Both effects make our finding that rare tests dominate the misses conservative. Targets in one module often share polluters, so the 39 executed targets are not independent confirmations.

Theory. Theorem 1 covers every S1 model in the dataset; this is a statement about the recorded models. The general class-contiguous case remains a conjecture, checked exhaustively up to seven tests. The proofs are complete, and the package checks their consequences exhaustively on small instances (14,791 hierarchy models, 731,630 orders) and verifies the budget and hitting-time identities in exact arithmetic.

External. The simulation covers 47 Maven modules and one tool at one commit. Execution covers feasibility-selected modules in one Linux/JDK environment. Other frameworks also nest tests (e.g., describe blocks in Jest [5]), and Theorem 1(a) covers arbitrary nesting, but the empirical magnitudes may differ.

VIII. RELATED WORK

OD test detection. DTDetector introduced reversal, randomised, and bounded exhaustive detection [3]. iDFlakies randomises class-contiguous orders and classifies flaky tests [4]. Other work detects state pollution or test dependencies directly [10]–[12], repairs OD tests [14], or detects them incrementally [15]. Empirical studies cover Python [16] and JavaScript [5]; see Parry et al. [2] for a survey. Systematic approaches cover test pairs with Tuscan squares [6], [8]. Li et al. found that most generated orders are unnecessary and called for prioritisation [8], which Iqbal et al. pursue [13].

Probabilistic analysis of test orders. Wei et al. derived flake rates for special cases and proposed the reversal change [6]. Wang et al. count failing orders with Alloy [17]. We prove the reversal inequality in general hierarchies, bound its value, and separate per-test from suite-level claims.

Antithetic sampling. Pairing a sample with a reflected one is the classical antithetic technique [18], [19], and its use for estimating probabilities is well studied [20]. Those results reduce estimator variance, and they help only when the pair is negatively correlated. Theorem 1 establishes that sign for OD-test semantics, using a certificate for hierarchies and Harris’ correlation inequality [9] for unconstrained orders. The sign is not generic (Table II). Theorem 2 and Eq. (3) concern detection times rather than estimator variance.

IX. CONCLUSION

Reversal’s value is bounded, even when its intuition is correct: independent pairs save at most half a run per target’s first failure, and independent small blocks cannot deliver constant-factor savings for rare failures. Under explicit OD-model hypotheses, pairing is safe; a suite-level gate is a different policy and does not inherit that guarantee. Simulation of 289 published models finds gains below 1 pp from 10 runs on. Real executions of three modules match all 40,200 predicted outcomes of 39 targets, including rarely failing ones, and the real iDFlakies follows the simulated gate rule in every decision. Our predictor tradeoff makes the next question concrete: can useful failure information be obtained accurately and cheaply enough to outperform outcome-blind sampling?

ACKNOWLEDGMENT

We thank GPT and Codex (OpenAI) and Claude (Anthropic), which helped us push the theory forward. They carried out much of the mathematical derivation for cases (b)–(d) of the safety theorem, wrote most of the simulation, verification, and execution code, and helped draft and polish the text. Gemini (Google) helped with the literature search. We then checked every proof and the idea behind it ourselves, cross-checked the derivations with independent exact computations, verified every citation against its source, and regenerated all numbers from the released scripts. We take full responsibility for the content.

DATA AVAILABILITY

The anonymized replication package is available at <https://anonymous.4open.science/r/tset-71C0>. It contains the complete proofs, exact verification scripts, the suite simulator, the analysis scripts that generate every number, table, and figure in this paper, the real-execution protocol, runner, and measurements, and the processed dataset of Li et al. [8] with provenance.

REFERENCES

- [1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” in *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 643–653.
- [2] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “A survey of flaky tests,” *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 1, pp. 17:1–17:74, 2021.
- [3] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, “Empirically revisiting the test independence assumption,” in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 385–396.
- [4] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, “iDFlakies: A framework for detecting and partially classifying flaky tests,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2019, pp. 312–322.
- [5] N. Hashemi, A. Tahir, S. Rasheed, A. Shi, and R. Blagojevic, “Detecting and evaluating order-dependent flaky tests in JavaScript,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2025.
- [6] A. Wei, P. Yi, T. Xie, D. Marinov, and W. Lam, “Probabilistic and systematic coverage of consecutive test-method pairs for detecting order-dependent flaky tests,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 12651. Springer, 2021, pp. 270–287.
- [7] UT-SE-Research, “iDFlakies source code, commit f54b3f027fd39f3ecc510b8459a44ba9bea3d839 (RandomDetector.java, TestShuffler.java),” <https://github.com/UT-SE-Research/iDFlakies>, accessed September 2026.
- [8] C. Li, M. M. Khosravi, W. Lam, and A. Shi, “Systematically producing test orders to detect order-dependent flaky tests,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 627–638.
- [9] T. E. Harris, “A lower bound for the critical probability in a certain percolation process,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 56, no. 1, pp. 13–20, 1960.
- [10] A. Gyori, A. Shi, F. Hariri, and D. Marinov, “Reliable testing: Detecting state-polluting tests to prevent test dependency,” in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2015.
- [11] J. Bell, G. Kaiser, E. Melski, and M. Dattatreya, “Efficient dependency detection for safe Java test acceleration,” in *Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015.
- [12] A. Gambi, J. Bell, and A. Zeller, “Practical test dependency detection,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2018.
- [13] H. Iqbal, Z. Begum, and K. Sakib, “Reduction of test re-runs by prioritizing potential order dependent flaky tests,” in *IEEE/ACM International Flaky Tests Workshop (FTW)*, 2025, arXiv:2510.26171.
- [14] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, “iFixFlakies: A framework for automatically fixing order-dependent flaky tests,” in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 545–555.
- [15] C. Li and A. Shi, “Evolution-aware detection of order-dependent flaky tests,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 114–125.
- [16] M. Gruber, S. Lukaczyk, F. Kroiß, and G. Fraser, “An empirical study of flaky tests in Python,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 148–158.
- [17] W. Wang, P. Yi, S. Khurshid, and D. Marinov, “Initial results on counting test orders for order-dependent flaky tests using Alloy,” in *Testing Software and Systems (ICTSS)*, ser. LNCS. Springer, 2021.
- [18] J. M. Hammersley and K. W. Morton, “A new Monte Carlo technique: Antithetic variates,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 52, no. 3, pp. 449–475, 1956.
- [19] A. B. Owen, *Monte Carlo Theory, Methods and Examples*, 2013, chapter 8, antithetic sampling. <https://artowen.su.domains/mc/>.
- [20] T. A. B. Snijders, “Antithetic variates for Monte Carlo estimation of probabilities,” *Statistica Neerlandica*, vol. 38, pp. 55–73, 1984.